Lecture 8

16 October 2026 20:36

Now the query and the key will interact with each other

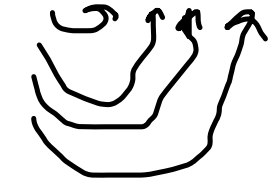
Query = what am I looking for ?
Key = what do I contain

```
# Version 4 : the Query and Key interaction
torch.manual_seed(1337)
B, T, C = 4, 8, 32  # Batch, Time, Channels
x = torch.randn(B,T,C)

tril = torch.tril(torch.ones(T, T))
wei = torch.zeros((T, T))
wei = wei.masked_fill(tril == 0, float("-inf"))
wei = F.softmax(wei, dim=-1)
out = wei @ x
```

Now we have 1 query ---> and many keys

Q1 k1 k2 k3 k4



The Q1 will interact with : k1
k2
and so on, ...

And give output, this output will train the model, as it will help the model to know which Query i.e the token is suitable for which predicted token, this will happen with every toekn (Q = query) and in every batch

We will do DOT PRODUCT of query and the key = and this will be equal to wei

THE WHOLE DIMENSION conversion:

STEP 1: Input

---

idx shape: (B, T) = (4, 8)
Contains raw integer token IDs

STEP 2: Token Embedding Lookup

---

token_embedding_table: shape: (vocab_size, n_embd) = (65, 32)

tok_emb = token_embedding_table(idx)
shape: (B, T, n_embd) = (4, 8, 32)
Each token ID → replaced by 32 features

-----> 32 features for each token ID's will be get as output for now

Hence, final size will be :
4 batch, 8 tokens and each with 32 features

STEP 3: Position Embedding Lookup

---

position_embedding_table: shape: (block_size, n_embd) = (8, 32)

torch.arange(T) = [0, 1, 2, 3, 4, 5, 6, 7]
shape: (T) = (8,)

pos_emb = position_embedding_table(torch.arange(T))
shape: (T, n_embd) = (8, 32)
Each position index → replaced by 32 features

Design will be:

Position ----> Tokens

i.e. 8 positions for 8 tokens ---> with 32 features already
hence, (8, 32)

STEP 4: Add Token + Position Embeddings

---

x = tok_emb + pos_emb shape: (B, T, n_embd) = (4, 8, 32)
PyTorch auto-broadcasts pos_emb (8,32)
across all 4 batches

As we want final (4, 8, 32)
So batch should be included,

When token and positions are added we get ---> (4, 8, 32)

STEP 5: Self-Attention (coming soon!)

---

x = self_attention(x) shape: (B, T, n_embd) = (4, 8, 32)

Shape stays the same!
But now each token knows its context

STEP 6: lm_head Linear Layer

---

lm_head: shape: (n_embd, vocab_size) = (32, 65)

logits = lm_head(x) shape: (B, T, vocab_size) = (4, 8, 65)
32 rich features → 65 character scores

STEP 7: Loss Calculation (during training only)

---

logits.view(B*T, vocab_size) shape: (32, 65)
targets.view(B*T) shape: (32,)
loss = F.cross_entropy(...) shape: scalar (single number e.g. 2.45)

STEP 8: Generation (during inference only)

---

logits[:, -1, :] shape: (B, vocab_size) = (4, 65)
probs = F.softmax(...) shape: (B, vocab_size) = (4, 65)
idx_next = multinomial(...) shape: (B, 1) = (4, 1)
idx = cat(idx, idx_next) shape: (B, T+1) = (4, 9)

At a Glance:

Step	Operation	Input Shape	Output Shape	
1	Raw input	—	(4, 8)	Step 4 is V IMP
2	Token embedding	(4, 8)	(4, 8, 32)	
3	Position embedding	(8,)	(8, 32)	
4	Add both	(4,8,32) + (8,32)	(4, 8, 32)	
5	Self-Attention	(4, 8, 32)	(4, 8, 32)	
6	lm_head	(4, 8, 32)	(4, 8, 65)	
7	Loss	(32, 65) + (32,)	scalar	
8	Generate	(4, 65)	(4, 1) new token	

In the version 4 we are creating the head, just initializing the query and the key

Up until version 2, tokens were **completely deaf and blind to each other**. Each token was just predicting the next character using only its own embedding:

- Token 3 didn't know what Token 1 or Token 2 were.

Why Version 4 is the turning point:

Version 4 is the exact moment **tokens start talking to each other based on their content**:

1. **Query (Q) & Key (K) interact:** $wei = Q \cdot K^T$ Every token computes an affinity score with every previous token: *"How relevant are you to what I need?"*
2. **Masking & Normalization:** Tokens only look into the past (not the future), and we softmax the scores so they act as weights summing to 1.
3. **Value (V) collects the info:** $out = wei \cdot V$ Instead of just averaging tokens blindly, tokens take a **weighted sum** of the most relevant tokens.

By building a **single head** first, we keep the code minimal and focus purely on watching this $Q \times K$ interaction come to life.

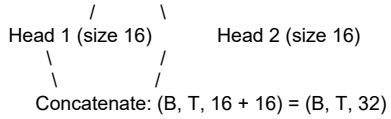
1. The Big Reason: Preparing for Multi-Head Attention 🧩

In a Transformer, we rarely use just one attention head. We use **multiple heads working in parallel**.
Look at how the math connects: $head_size = \frac{n_embd}{n_head} = \frac{32}{2} = 16$
If we have **2 heads**, each projecting to size **16**:

1. **Head 1** outputs a vector of size 16
2. **Head 2** outputs a vector of size 16
3. We **concatenate** them side-by-side: 16 + 16 = 32

Now the output is back to shape (B, T, 32) — perfectly matching n_embd!

Input x: (B, T, 32)
/ \



2. Why not just 1 big head of size 32? (Specialization) 👥

Think of it like dividing work:

- **One 32-dim head:** Only one perspective. The tokens can only communicate about one relationship at a time.
- **Two 16-dim heads:** Two different "conversations" happening simultaneously for the same total compute:
  - **Head 1 (16 dims):** Might learn grammatical relationships (e.g., *"where is the subject of this verb?"*).
  - **Head 2 (16 dims):** Might learn punctuation or quotation tracking (e.g., *"are we inside quote marks?"*).

3. Efficiency & Bottleneck

Tokens don't need all 32 dimensions just to compute similarity scores:

- When calculating affinity: $scores = Q \times K^T$ Using 16 dimensions keeps the dot product lighter and acts as a **feature bottleneck**, forcing the query and key to retain only the most essential information needed to find relevant tokens.

Summary

In version 4, you start with a **single head of size 16** just to understand the mechanics. Immediately right after, you'll group multiple heads together so they add back up to 32!

Yes, exactly! We are creating Just 1 single head for now.

We do **not** jump into multi-head attention yet.
The goal right now is to master the mechanics of **one head** first:

1. Compute Q (Query)
2. Compute K (Key)
3. Compute V (Value)
4. Score them: $\frac{Q \cdot K^T}{\sqrt{d_k}}$
5. Mask out the future (triangular mask)
6. Apply Softmax to get probabilities
7. Multiply probabilities by V

V IMP

Part 1: What nn.Linear does here

python
```
head_size = 16  # smaller than n_embd=32

self.query = nn.Linear(n_embd, head_size, bias=False)     # 32 → 16
self.key   = nn.Linear(n_embd, head_size, bias=False)      # 32 → 16
self.value = nn.Linear(n_embd, head_size, bias=False)     # 32 → 16
```

nn.Linear(32, 16) is just a **learned weight matrix** W of shape (16, 32).
When you call it:

python
```
q = self.query(x)  # (B, T, 32) @ W_Q^T → (B, T, 16)
k = self.key(x)    # (B, T, 32) @ W_K^T → (B, T, 16)
```
It **projects** each token's 32-dim vector into a smaller 16-dim **"query space"** and **"key space"**.

Part 2: Why separate projections?

Without Q/K linear layers, you'd just do x @ x.T — every token talks to itself in the **same space**. That's too rigid.

With separate nn.Linear layers:

- W_Q teaches *"how to formulate a good question"*
- W_K teaches *"how to advertise your contents"*

They are **learned independently** — the model figures out the best way to match questions to answers during training.

Part 3: The full attention flow in v4

```
x → W_Q → Q  (what I want)
x → W_K → K  (what I have)
x → W_V → V  (what I send)
scores = Q @ K^T        # (B, T, T) — how much does each token's Q match each token's K?
scores = scores / √head_size  # scale to avoid exploding softmax
scores = tril mask      # can't look into the future!
```

```
weights = softmax(scores)
out = weights @ V       # weighted sum of Values
```

Analogy 🔎

Think of a **YouTube search**:

- Your **Query** = the search text you typed
- Each video's **Key** = its title/tags
- Each video's **Value** = the actual video content

nn.Linear for Q and K is learning *how to encode good search queries and good tags* — two different skills, trained separately.

So when you write v4's Head class, it'll look like:

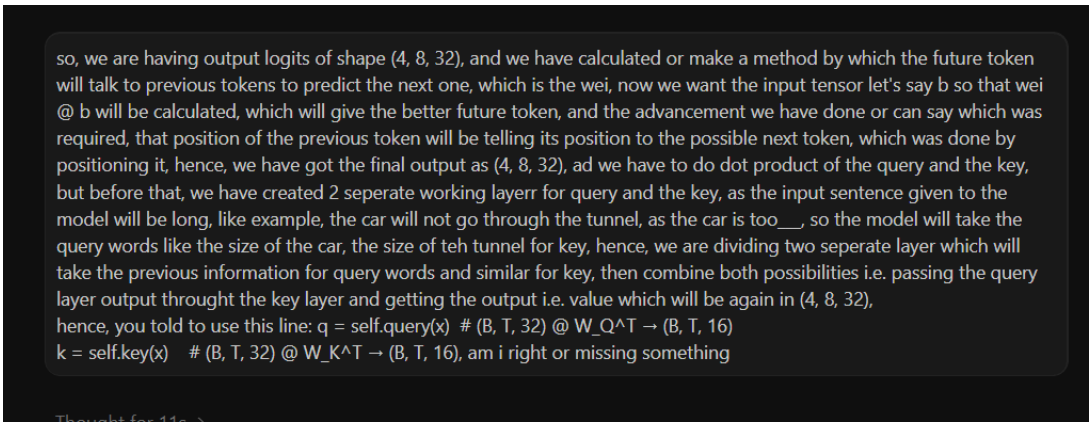
Python

```
class Head(nn.Module):
   def __init__(self, head_size):
      super().__init__()
      1. self.key   = nn.Linear(n_embd, head_size, bias=False)         # learned "advertise"

      2. self.query = nn.Linear(n_embd, head_size, bias=False)       # learned "question"

      3. self.value = nn.Linear(n_embd, head_size, bias=False)         # learned "output"

      4. self.register_buffer('tril', torch.tril(torch.ones(block_size, block_size)))
```



This will tell us, how have we understand the self-attention concept:

Answer by antigravity :
You have grasped the **core intuition amazingly well**—especially your **car and tunnel example** (that is literally how attention solves long-range context!).

However, there are **two small but crucial misconceptions** in how Query, Key, and Value actually interact in the code. Let's iron them out:

1. You got this right: The Car & Tunnel Analogy 🚗 🚇

> *"The car will not go through the tunnel, as the car is too ___"*

At the blank (___):

- **Query (Q):** The token at ___ asks: *"I need an adjective describing physical size."*
- **Key (K):** The earlier token car advertises: *"I am an object with size properties!"*
- When Q and K match, their dot product is large.

2. Misconception #1: Query does NOT pass *through* the Key layer

You wrote:

> *"passing the query layer output through the key layer"*

They are **not sequential**; they run in **parallel** on the same input x:

```
          x  (B, T, 32)
         /  \
        /    \
 self.query(x)   self.key(x)
     ↓              ↓
  q (B, T, 16)    k (B, T, 16)
```

V. Imp

They interact via a **matrix dot product** to create wei (the affinities): $wei = q \quad @ \quad k^T$ *Shape*: $(B, T, 16) \times (B, 16, T) \longrightarrow (B, T, T)$
(B, T=8, head divided for particular query = 16) (B, head divided for particular key = 16, T = 8)

B = batch will remain same (T=8, H=16) @ (H=16, T=8) ----> (T=8, T=8) ---> matrix multiplication only happen when column of the first matrix = row of the second matrix

wei is **NOT** the final value. wei is just a table of numbers representing: *"How much does token i care about token j?"*

V IMP:

3. Misconception #2: What is the "Value" (V)?

You wrote:

> *"now we want the input tensor let's say b so that wei @ b will be calculated... getting the output i.e. value"*

You hit the nail on the head when you said *"we want an input tensor let's say b so that wei @ b will be calculated"*.

That tensor b IS the Value (V)!

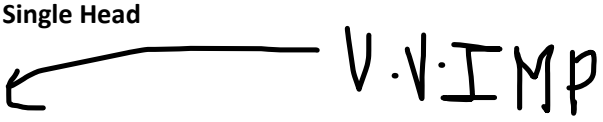
- In version 3 (the naive averaging version), we did: $out = wei \quad @ \quad x$ Here, tokens literally averaged raw embeddings x.
- In version 4 (Self-Attention), we don't pass raw x. Instead, we pass x through a **third linear layer**: $v = self.value(x) \quad shape \quad (B, T, 16)$

So the Value is **not** produced by combining Q and K. Value is a third separate projection of x:

- Q: *"What am I looking for?"*
- K: *"What do I have to offer?"*
- V: *"Here is the actual information I will send if you choose me!"*

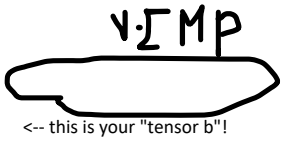
V V IMP:

Summary Diagram: The Entire Single Head




```
            x : (B, T, 32)
          /      |      \
         /       |       \
self.query(x)  self.key(x)  self.value(x)
    ↓              ↓            ↓
 q (B, T, 16)   k (B, T, 16)  v (B, T, 16)
    \            /                |
     \          /                 |
    wei = q @ k.T                 |
    Shape: (B, T, T)              |
        ↓                         |
    mask + softmax                |
        ↓                         |
    wei (B, T, T)                 |
        \                        /
         \                      /
          out = wei @ v
          Shape: (B, T, 16)
```

The 5 lines in code:




```
# 1. Project into Q, K, V
q = self.query(x)  # (B, T, 16)
k = self.key(x)    # (B, T, 16)
v = self.value(x)  # (B, T, 16)        <-- this is your "tensor b"!

# 2. Compute attention scores ("affinities")
wei = q @ k.transpose(-2, -1) * (16 ** -0.5)  # (B, T, T)

# 3. Prevent looking into the future

wei = wei.masked_fill(self.tril[:T, :T] == 0, float('-inf'))
wei = F.softmax(wei, dim=-1)

# 4. Weighted aggregation of values
out = wei @ v  # (B, T, T) @ (B, T, 16) --> (B, T, 16)
```

Notice the output of this single head is **(B, T, 16)** (not 32).

ONE MORE EXAMPLE:

from the summary diagram, i understood that, the query and key will help only to give what i have and what am i looking for, and output of them, combine with the possible values v will give a high score for that answer

BINGO! You've nailed it completely. 🎯
That is the exact philosophy of Attention.
To make it 100% concrete, think of wei as a **volume knob** and V as the **audio content**:

1. Q **and** K **set the volume knob (wei):**
   - The token at "too ___" matches strongly with the key for "car".
   - So wei turns the volume for "car" up to **90%** and the volume for "the" down to **1%**.
2. V **is what actually plays through the speaker (out = wei @ v):**
   - $$out = 0.90 \times V_{car} + 0.01 \times V_{the} + \ \ldots$$
   - Because "car" had the highest score, its Value vector **dominates** the output.

- Now, the position "too ___" is packed with rich information about the car's size, ready to predict the next word ("big", "wide").

Till now the code look like this:

```
# Version 4: Self-attention
torch.manual_seed(1337)

B, T, C = 4, 8, 32
x = torch.randn(B, T, C)

# Let's see a single head perform the Self Attention
head_size = 16

# V IMP : Linear helps to create two separate Query and Key layer in neural network
key = nn.Linear(C, head_size, bias=False)   # Either pass C (channels) or the n_embd as both now are 32 in size
query = nn.Linear(C, head_size, bias=False)

# V IMP
k = key(x)   # (B, T=8, C=16)  as we have divided into two parts, hence, H = size of the single head = 16
q = query(x)   # (B, T=8, C=16)
wei = q @ k.transpose(-2, -1)   # matrix-multiply   (B=4, T=8, H=C=16) @ (B=4, H=C=16, T=8) ---> (B, T, T)

tril = torch.tril(torch.ones(T, T))
#wei = torch.zeros((T, T))
#wei = wei.masked_fill(tril == 0, float("-inf"))
#wei = F.softmax(wei, dim=-1)
out = wei @ x
```

Printing wei

Output:

```
tensor([[[1.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
         [0.1574, 0.8426, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
         [0.2088, 0.1646, 0.6266, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
         [0.5792, 0.1187, 0.1889, 0.1131, 0.0000, 0.0000, 0.0000, 0.0000],
         [0.0294, 0.1052, 0.0469, 0.0276, 0.7909, 0.0000, 0.0000, 0.0000],
         [0.0176, 0.2689, 0.0215, 0.0089, 0.6812, 0.0019, 0.0000, 0.0000],
         [0.1691, 0.4066, 0.0438, 0.0416, 0.1048, 0.2012, 0.0329, 0.0000],
         [0.0210, 0.0843, 0.0555, 0.2297, 0.0573, 0.0709, 0.2423, 0.2391]],

        [[1.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
         [0.1687, 0.8313, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
         [0.2477, 0.0514, 0.7008, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
         [0.4410, 0.0957, 0.3747, 0.0887, 0.0000, 0.0000, 0.0000, 0.0000],
          and so on, ……..
```

1. What is the use of the masked function:

It will provide with real raw outputs from the dot product of the query and the key, and as we want that the token at the 5 position will not look at the 6 token, which will be converted to 0.0 using -inf

Remaining same code

```
tril = torch.tril(torch.ones(T, T))
#wei = torch.zeros((T, T))
wei = wei.masked_fill(tril == 0, float("-inf"))      ---> not commented now
#wei = F.softmax(wei, dim=-1)
out = wei @ x
```

Print wei

Output:

```
tensor([[[-1.7629e+00,      -inf,      -inf,      -inf,      -inf,
                -inf,      -inf,      -inf],
         [-3.3334e+00, -1.6556e+00,      -inf,      -inf,      -inf,
                -inf,      -inf,      -inf],
         [-1.0226e+00, -1.2606e+00,  7.6228e-02,      -inf,      -inf,
                -inf,      -inf,      -inf],
         [ 7.8359e-01, -8.0143e-01, -3.3680e-01, -8.4963e-01,      -inf,
                -inf,      -inf,      -inf],
         [-1.2566e+00,  1.8719e-02, -7.8797e-01, -1.3204e+00,  2.0363e+00,
                -inf,      -inf,      -inf],
         [-3.1262e-01,  2.4152e+00, -1.1058e-01, -9.9305e-01,  3.3449e+00,
```

2. Now softmax it

It will help to give the real probability of what am I looking for --> the Query and the, what do I have or contain ---> Key

Remaining same code

```
tril = torch.tril(torch.ones(T, T))
#wei = torch.zeros((T, T))
wei = wei.masked_fill(tril == 0, float("-inf"))      ---> not commented now
wei = F.softmax(wei, dim=-1)                         ---> not commented now
out = wei @ x
```

Print wei

Output:

```
tensor([[[1.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
         [0.1574, 0.8426, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
         [0.2088, 0.1646, 0.6266, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
         [0.5792, 0.1187, 0.1889, 0.1131, 0.0000, 0.0000, 0.0000, 0.0000],
         [0.0294, 0.1052, 0.0469, 0.0276, 0.7909, 0.0000, 0.0000, 0.0000],
         [0.0176, 0.2689, 0.0215, 0.0089, 0.6812, 0.0019, 0.0000, 0.0000],
```

```
        [0.1691, 0.4066, 0.0438, 0.0416, 0.1048, 0.2012, 0.0329, 0.0000],
        [0.0210, 0.0843, 0.0555, 0.2297, 0.0573, 0.0709, 0.2423, 0.2391]],

       [[1.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
        [0.1687, 0.8313, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
```